

What Crosses the Wire

What the add-in reads inside a Word document, how it retrieves an official source, and what never leaves the workstation.

Audience – Information security review **Subject** – Ibid, a Word task-pane add-in

Verifiability – every claim cites the source that proves it

- 01 **The analysis is entirely local.** Citation recognition runs inside the Word task pane itself, on the workstation. No server takes part in it, and nothing is transmitted to accomplish it.
- 02 **Exactly one request leaves the workstation,** and only when the reviewer selects a citation. It carries the public identifier of the cited decision: nine fields, all drawn from public EU nomenclature.
- 03 **The add-in cannot modify the document.** It requests the Word permission `ReadDocument`, which is read-only. No write call exists anywhere in the source, and Word would not grant it the right.

§1 What the add-in analyses

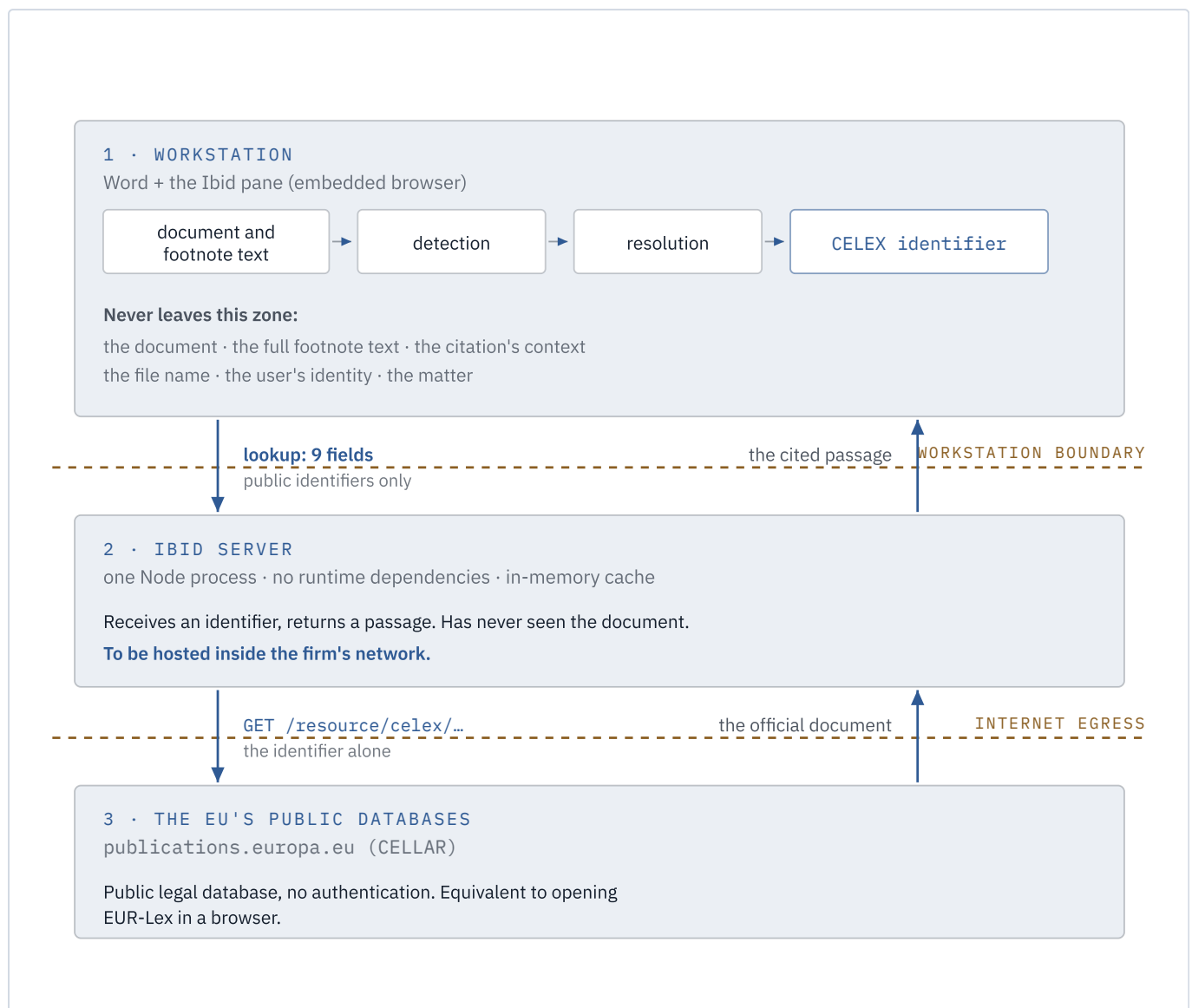
Ibid is a reading aid. It identifies references to EU law in footnotes — ECLI identifiers, Court of Justice and General Court case numbers, directives, regulations, Treaty articles, DG Competition case numbers — and displays the corresponding official passage alongside them.

It reads the full text of the open document, and we state that plainly rather than let you infer that it only touches footnotes. Two things make it necessary.

First, short-form citations mean nothing in isolation. A footnote reading "*Akzo Nobel*, para. 40" carries no information on its own; it only means something relative to a full citation stated earlier. Ibid therefore reads the document in order so it can resolve each later reference back to the authority it points at. Second, the pane follows the cursor: putting the caret on a reference mark brings up that citation's source, which requires knowing where the caret is in the body.

That reading happens in memory, inside Word's embedded browser (WebView2 on Windows, WKWebView on macOS). The recognition module is compiled into the pane's own JavaScript bundle and executes on the workstation. Detecting citations, resolving short forms, following chains of back-references — `Ibid.`, `supra` — and deriving the CELEX identifier: the whole of that chain is local and issues no request.

Three zones, and what passes between them



The document is read and analysed in zone 1 and does not leave it. Only the cited decision's identifier crosses the first boundary, and only that same identifier crosses the second. Host the server inside the firm's network and the first boundary becomes an internal one — nothing leaves the firm but a query to a public database.

How it retrieves the source

Once a citation has been recognised and normalised locally, the pane holds an official identifier — the CELEX number, the public nomenclature of the EU Publications Office. Fetching the text is then a matter of requesting a public document by its number.

That happens through a single request, issued from a single line of code, and only when the reviewer selects a citation:

```
# addin/src/ui/App.tsx – the pane's only network call
GET {ibid-origin}/api/sources?lookup={...}
```

The lookup object contains these fields and nothing else.

FIELD	EXAMPLE	WHAT IT IS
source	curia	Which database to query
value	C-293/12	The citation as written in the footnote
celex	62012CJ0293	Official EU identifier
ecli	ECLI:EU:C:2014:238	European Case Law Identifier
caseNumber	C-293/12	Case number
caseName	Digital Rights Ireland	Published case name
documentType	judgment	Judgment, opinion or order
locator	{ kind: 'point', start: 40 }	The point cited
paragraphs	[40]	Every paragraph cited

The server then requests the document from the Publications Office by that identifier, extracts the passage matching the cited point, and returns it. At no stage does it know which document the citation came from.

STATED PRECISELY, BECAUSE A REVIEWER WILL CHECK

Two of these fields are substrings of the footnote: `value`, the citation exactly as written, and `caseName`, the case name where one is recognised. Those are the reference itself — the thing being looked up — and no query can happen without them. What never crosses the boundary is everything *around* the citation: the sentence carrying it, the argument it supports, the rest of the footnote, the rest of the document.

§4 What never crosses, and why that is structural

✗ The document

✗ The footnote as written

✗ The citation's context

✗ The file name

✗ The user's identity

✗ Anything about the matter

The exclusion is not incidental. It is guaranteed by the way the request is built — and it is the point we would invite your team to verify first.

The object handed to the lookup function is a `CitationContext`, defined as `CitationMatch & { context: string }`. In other words: **the surrounding prose is present on the object, in memory, at the moment the request is formed.** A single object spread would have sent it along with everything else.

Instead the request is assembled by naming its nine fields one at a time, with no spread operator. The consequence is durable: a developer who adds a field to the citation type tomorrow cannot cause it to start crossing the wire by accident — they would have to type it into the lookup explicitly. That is the difference between "context is not sent today" and "context is not sent."

```
addin/src/ui/App.tsx — const lookup = { ... } · shared/src/index.ts — CitationContext
```

§5 Permissions, storage, retention

Word permission. The manifest declares `ReadDocument`, read-only. No write call — `insertText`, `insertParagraph`, `insertHtml`, `insertOoxml` — appears anywhere in the pane's source. The add-in is structurally incapable of altering a document.

Accounts. None. No authentication, no sign-in, no user identifier of any kind.

Client-side storage. The pane uses neither `localStorage` nor `sessionStorage`, and sets no cookies.

Server-side storage. An in-memory cache of retrieved passages, lost when the process restarts. Nothing is written to disk: the server contains no file-write call at all.

Logging. The server logs its own startup and startup failures. It does not log requests.

Telemetry. None. No analytics, no error reporting, no advertising, and no fonts or scripts served from any third-party CDN.

§6 Software supply chain

COMPONENT	RUNTIME DEPENDENCIES
The server	None. <code>api/package.json</code> declares no runtime dependencies at all. It runs on the Node standard library — <code>node:http</code> and the built-in <code>fetch</code> . TypeScript and ESLint are build-time only.
The Word pane	Two: <code>react</code> and <code>react-dom</code> . Everything else is a development dependency and is not shipped.

The shipped bundle is therefore this repository's own source plus React. There is no component library, no analytics SDK, and no telemetry agent to audit.

§7 Two points we put to you ourselves

A note like this is only worth reading if it also flags what deserves discussion. There are two, and we would rather raise them than have you find them.

POINT 1 — REVERSE-PROXY ACCESS LOGS

The lookup travels in the query string of a `GET`. Any reverse proxy or load balancer placed in front of the service — Caddy, nginx — will therefore record it in its access log by default. The application logs nothing; the infrastructure might. Two remedies, either of which works: disable access logging on the `/api` route, or change the endpoint to accept `POST`. We are glad to make that change if you would prefer it.

POINT 2 — A LOOKUP IS STILL INTELLIGENCE

Ibid does not transmit client material. It does transmit *which authorities are being researched, and when*, to whichever host runs the service. That is not privileged content, but it is not nothing either — and it is precisely the fact on which the hosting decision should turn.

What follows from those two points: where to host

RECOMMENDED

Inside the firm's infrastructure

Nothing leaves the firm's control except the onward query to a public legal database. The server is a single Node process with no dependencies and no persistent state.

EVALUATION

Hosted externally

Appropriate for an evaluation period, provided the service stays stateless and access logging is switched off.

THEORETICAL

On the reviewer's machine

Nothing leaves at all, but it requires Node and a locally trusted certificate — generally impractical on a managed device.

§8 Verify rather than take our word

Every claim in this note is checkable against the repository. These commands produce exactly the result stated beside them.

```
# One outbound request in the pane, and no browser storage.
# Expected: a single hit, in App.tsx.
grep -rn "fetch(\|XMLHttpRequest\|sendBeacon\|WebSocket\|localStorage\|sessionStorage" addin/src/
shared/src/

# The add-in cannot write to the document. Expected: no hits.
grep -rn "insertText\|insertParagraph\|insertHtml\|insertOoxml" addin/src/

# Read-only permission. Expected: <Permissions>ReadDocument</Permissions>
grep -n "Permissions" addin/manifest.xml

# Exactly what is put on the wire: nine named fields, no spread.
sed -n '/const lookup = {/,/};/p' addin/src/ui/App.tsx

# Nothing is written to disk, anywhere in the server. Expected: no hits.
grep -rn "node:fs\|writeFile\|createWriteStream\|appendFile" api/src/ api/server.mjs

# The server logs startup only, never requests.
grep -n "console\." api/server.mjs

# Runtime supply chain: none for the API, react + react-dom for the pane.
grep -A4 '"dependencies"' api/package.json addin/package.json
```

The repository carries this note as `docs/DATA-FLOW.md`, alongside `docs/HOSTING.md` for deployment and for centralized distribution through the Microsoft 365 admin center. We are happy to walk a reviewer through the code directly.

Ibid. · What Crosses the Wire · technical note for information
security review

Verifiable against the repository at every
claim